

Concurrency in Go

Concurrency in Go is built on three pillars: goroutines (covered separately), channels, and the `sync` package. The philosophy is simple: share memory by communicating, instead of communicating by sharing memory.

Channels

A channel is a typed pipe. One goroutine sends, another receives. Sends and receives block until both sides are ready (for unbuffered channels).

```
ch := make(chan int)           // unbuffered
ch := make(chan int, 10)       // buffered with capacity 10

ch <- 42                       // send (blocks if no receiver / buffer full)
v := <-ch                      // receive (blocks if no sender / buffer empty)
v, ok := <-ch                 // ok is false if channel is closed and drained
close(ch)                     // sender closes; receivers can drain remaining values
for v := range ch { ... }     // ranges until channel is closed and drained
```

Unbuffered vs buffered

	Unbuffered	Buffered
Send blocks until	a receiver is ready	the buffer has space
Receive blocks until	a sender is ready	the buffer has a value
Use case	hand-off, synchronization	decoupling producer and consumer rates
Order guarantee	strict FIFO	strict FIFO

Rule of thumb: start unbuffered. Add buffering only when you can justify the capacity number with a real reason (rate smoothing, batch sizes, etc.). A random `make(chan T, 100)` is a code smell.

Closing rules

1. Only the sender closes a channel. Never close from the receiver side.
2. Closing a closed channel panics.
3. Sending on a closed channel panics.
4. Receiving from a closed channel returns the zero value immediately with `ok == false`.
5. If multiple senders exist, use a separate signal (a `done` channel or `sync.Once`) to coordinate the close.

Channel direction

You can constrain a channel parameter to send-only or receive-only. This makes intent explicit and prevents misuse.

```
func producer(out chan<- int) { out <- 1 } // send-only
func consumer(in <-chan int) { <-in }      // receive-only
```

A bidirectional channel converts to either direction implicitly. The reverse is not allowed.

Nil channels

A nil channel blocks forever on send and receive. This sounds useless, but it's the standard trick for disabling a `select` case dynamically:

```
var ch chan int // nil channel
select {
case v := <-ch: // never fires
case <-time.After(time.Second): // wins
}
```

Set a case's channel to `nil` inside a `select` loop to “turn off” that branch without breaking the loop.

The select statement

`select` waits on multiple channel operations. If several are ready, one is picked at random. If none are ready and there's no `default`, it blocks.

```
select {
case v := <-chA:
    handle(v)
case chB <- 42:
    sent()
case <-time.After(time.Second):
    timeout()
case <-ctx.Done():
    return ctx.Err()
default:
    // non-blocking branch when nothing is ready
}
```

Common select patterns

Timeout on a single operation:

```
select {
case v := <-ch:
    use(v)
case <-time.After(2 * time.Second):
    return errors.New("timeout")
}
```

Non-blocking send/receive (use `default`):

```
select {
case ch <- v:
    // sent
default:
    // dropped because no receiver
}
```

Cancellation via context:

```
for {
    select {
    case <-ctx.Done():
        return ctx.Err()
    case job := <-jobs:
        process(job)
    }
}
```

Disable a branch by setting its channel to nil:

```
var out chan<- Result
for {
    select {
    case r := <-results:
        out = pending           // enable the send branch
        next = r
    case out <- next:
        out = nil               // disable until next result arrives
    }
}
```

Select gotchas

- A bare `for { select { ... } }` with only a `default` case spins the CPU. Always include a blocking case.
- `time.After` inside a hot loop allocates a new timer every iteration. Use `time.NewTimer` and reset it, or pull the timer outside the loop.
- The choice between ready cases is pseudorandom. Don't rely on ordering.

Context

`context.Context` carries cancellation signals, deadlines, and request-scoped values across API boundaries. Pass it as the first argument to any function that does I/O, blocks, or spawns goroutines.

```
ctx, cancel := context.WithCancel(parent)
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
ctx, cancel := context.WithDeadline(parent, t)
defer cancel() // always defer cancel
```

Cancellation propagation

A cancelled parent cancels every child. A child that times out does not cancel its parent. Tree, not graph.

```
func fetch(ctx context.Context, url string) error {
    req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
    _, err := http.DefaultClient.Do(req) // aborts if ctx is cancelled
    return err
}
```

Context values

`context.WithValue` stores request-scoped data (request ID, auth token, trace span). Use typed keys to avoid collisions:

```
type ctxKey struct{}
var userIDKey = ctxKey{}

ctx = context.WithValue(ctx, userIDKey, "u-123")
id, _ := ctx.Value(userIDKey).(string)
```

Don't smuggle business parameters through `Value`. It's for cross-cutting metadata only.

Context rules

1. Pass `ctx` as the first argument, named `ctx`.
 2. Never store a `Context` in a struct. Pass it explicitly.
 3. Always call `cancel()`, ideally with `defer`. Leaked contexts leak goroutines.
 4. `context.Background()` for the root. `context.TODO()` only when you don't yet know what to use.
 5. Check `ctx.Err()` after a blocking call returns to distinguish cancellation from other errors.
-

sync primitives

The `sync` package is for cases where channels are awkward: protecting shared state, one-time initialization, waiting on N goroutines, etc.

sync.Mutex

A plain mutual exclusion lock. No re-entry, no recursion.

```
type Counter struct {
    mu sync.Mutex
    n   int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}
```

Always lock by pointer receiver. Copying a struct that contains a mutex is a bug (`go vet` catches this).

sync.RWMutex

Multiple concurrent readers, one writer.

```
mu.RLock()    // many at once
mu.RUnlock()
mu.Lock()     // exclusive
mu.Unlock()
```

Use when reads outnumber writes by a wide margin (think 100:1). Below that, plain `Mutex` is faster because `RWMutex` has higher overhead per call.

sync.WaitGroup

Wait for a known number of goroutines to finish.

```
var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    go func(it Item) {
        defer wg.Done()
        process(it)
    }(it)
}
wg.Wait()
```

Rules: 1. Call `Add` before the goroutine starts, not inside it. 2. `Done` must run even on panic (use `defer`). 3. Don't reuse a `WaitGroup` across overlapping batches. Build a new one.

sync.Once

Run a function exactly once, no matter how many goroutines call it.

```
var once sync.Once
var conn *DB

func getDB() *DB {
    once.Do(func() {
        conn = connect()
    })
    return conn
}
```

If the function inside `Do` panics, the `Once` is still considered “done”. Subsequent calls won't retry.

sync.Cond

Conditional waiting on a predicate. Rarely needed in Go. Most cases are better served by channels.

```
c := sync.NewCond(&sync.Mutex{})
c.L.Lock()
for !ready {
    c.Wait()
}
c.L.Unlock()
// elsewhere:
c.L.Lock(); ready = true; c.L.Unlock(); c.Broadcast()
```

If you find yourself reaching for `Cond`, ask whether a channel works first.

sync.Pool

A pool of reusable objects to cut allocation pressure in hot paths.

```
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

buf := bufPool.Get().(*bytes.Buffer)
buf.Reset()
defer bufPool.Put(buf)
```

Caveats: - Items can be evicted between GC cycles. The pool is best-effort. - Always `Reset` after `Get` and before `Put`. - Don't pool objects that hold references to large memory; the eviction surprise can leak.

sync.Map

A concurrent map for two specific access patterns: keys written once and read many times, or disjoint key sets across goroutines.

```
var m sync.Map
m.Store("k", 1)
v, ok := m.Load("k")
m.Delete("k")
m.Range(func(k, v any) bool { return true })
```

For most cases, a plain `map` with a `sync.RWMutex` is simpler and faster. Reach for `sync.Map` only when benchmarks show contention on the plain version.

atomic package

For lock-free integer and pointer updates. Useful for counters, flags, and CAS loops.

```
import "sync/atomic"

var n atomic.Int64
n.Add(1)
v := n.Load()
n.Store(42)
swapped := n.CompareAndSwap(0, 1)
```

Go 1.19+ added typed wrappers (`atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]`). Prefer these over the older `atomic.AddInt64(&x, 1)` style.

Atomics give the strongest happens-before guarantee on the variable being touched, but they don't synchronize anything else. Mixing atomic and non-atomic access on the same variable is undefined behavior.

errgroup

`golang.org/x/sync/errgroup` is `WaitGroup` plus error propagation plus context cancellation. Use it for fan-out work that can fail.

```
import "golang.org/x/sync/errgroup"

g, ctx := errgroup.WithContext(ctx)
for _, url := range urls {
    url := url
    g.Go(func() error {
        return fetch(ctx, url)
    })
}
if err := g.Wait(); err != nil {
    return err
}
```

When any goroutine returns an error, `ctx` is cancelled. The other goroutines see `ctx.Done()` and stop. `g.Wait()` returns the first error.

`g.SetLimit(n)` (Go 1.20+) caps the number of in-flight goroutines.

Memory model essentials

The Go memory model defines when one goroutine is guaranteed to see another's writes. Key happens-before relationships:

1. A send on a channel happens before the corresponding receive completes.
2. A receive on a closed channel happens after the close.
3. `Mutex.Unlock` happens before any subsequent `Lock` returns.
4. `Once.Do(f)` returns after `f` completes (visible to all callers).
5. `WaitGroup.Wait` returns after the matching `Done` calls.
6. The `go` statement happens before the goroutine starts.

Without one of these, there's no guarantee that goroutine B sees what goroutine A wrote. The race detector catches violations at runtime; the compiler doesn't.

Common concurrency patterns

Fan-out, fan-in

Split work across N workers, merge results into one channel.


```

func fanIn(chans ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(chans))
    for _, c := range chans {
        go func(c <-chan int) {
            defer wg.Done()
            for v := range c { out <- v }
        }(c)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}

```

Bounded worker pool

```

jobs := make(chan Job)
var wg sync.WaitGroup
for i := 0; i < runtime.NumCPU(); i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for j := range jobs { process(j) }
    }()
}
for _, j := range allJobs { jobs <- j }
close(jobs)
wg.Wait()

```

Pipeline

Stages connected by channels. Each stage runs in its own goroutine.

```

gen := func(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums { out <- n }
    }()
    return out
}

square := func(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in { out <- n * n }
    }()
    return out
}

for v := range square(gen(1, 2, 3)) { fmt.Println(v) }

```

Each stage closes its output channel when its input is drained. Cancellation propagates by closing upstream.

Semaphore via buffered channel

Cap the number of concurrent operations.

```

sem := make(chan struct{}, 5) // at most 5 concurrent

for _, item := range items {
    sem <- struct{}{}
    go func(it Item) {
        defer func() { <-sem }()
        process(it)
    }(item)
}

```

Rate limiting with a ticker

```

limit := time.NewTicker(100 * time.Millisecond)
defer limit.Stop()
for _, req := range requests {
    <-limit.C
    go handle(req)
}

```

For real production rate limiting, use golang.org/x/time/rate (token bucket).

Once-only initialization with retry

Plain `sync.Once` doesn't retry on failure. If you need that, use a mutex with a "done" flag and clear the flag on error.

Channels vs sync: when to use which

Reach for channels when: - The data flows from one goroutine to another (pipelines, queues, work distribution). - You're coordinating goroutine lifecycle (cancellation, completion signals). - The pattern naturally fits `select` (timeouts, fan-in, multiplexing).

Reach for `sync` (Mutex, atomic, WaitGroup) when: - You're protecting shared state inside a single struct. - A single counter or flag needs to be updated by many goroutines. - The overhead of a channel is too high for the operation.

Both are correct Go. The Rob Pike line "share memory by communicating" is a default, not a law. A `Mutex` around a map is often cleaner than a goroutine that owns the map and serves it via channels.

Race conditions

A race exists when two goroutines access the same memory and at least one of them writes, with no synchronization between them. The result is undefined.

Detect with `-race`

```
go test -race ./...
go run -race main.go
go build -race
```

The race detector instruments memory accesses at runtime. It catches actual races as they happen during execution. It does not find races that didn't trigger during the run.

Cost: 5-10x memory, 2-20x slower. Run `-race` in CI and during development, not production.

Common race patterns

Map written concurrently: Go's built-in map is not safe for concurrent writes. Even one writer with concurrent readers is a race. Wrap with `sync.RWMutex` or switch to `sync.Map`.

Loop variable in goroutine (pre-Go 1.22):

```
// BUG in Go < 1.22: all goroutines see the same i
for i := 0; i < 10; i++ {
    go func() { fmt.Println(i) }()
}
```

Fix by passing as argument or rebinding `i := i` inside the loop. Go 1.22 changed the semantics so each iteration gets a fresh `i`.

Forgotten lock: updating a field on the way in, reading it on the way out, locking only one path.

Deadlocks

A deadlock is when every goroutine is blocked waiting for another. The runtime detects total deadlock and panics:

```
fatal error: all goroutines are asleep - deadlock!
```

Partial deadlock (some goroutines deadlocked, others still running) is not detected. The program just hangs.

Common deadlock causes

1. **Unbuffered send with no receiver:** `ch <- 1` blocks forever if nothing reads.
2. **Closing then sending:** sending on a closed channel panics; closing twice panics.
3. **Forgotten close on a ranged channel:** `for v := range ch` blocks forever if the producer never closes.
4. **Lock ordering inversion:** goroutine A holds X and waits for Y; goroutine B holds Y and waits for X. Always acquire locks in a consistent global order.
5. **Self-receive:** a goroutine sending and receiving on its own unbuffered channel without `select`.
6. **WaitGroup misuse:** `wg.Wait()` before any `wg.Add`, or `Add` inside a goroutine that hasn't started yet.

How to avoid

- Define a clear ownership model. One goroutine owns the channel; only it closes.
 - Acquire locks in a fixed order across the codebase. Document the order.
 - Use `context` for cancellation so blocked goroutines have an escape hatch.
 - Set a timeout on every blocking operation that doesn't already have one.
 - Run with `-race` and `-gcflags=-d=checkptr` in tests.
-

Best practices

1. **Pass context first.** Every function that blocks or starts a goroutine takes `ctx context.Context` as the first parameter.
 2. **Never start a goroutine without knowing how it ends.** Every `go func()` should have a clear termination path (channel close, context cancel, finite work).
 3. **Don't leak goroutines.** A blocked goroutine holds its stack and any captured references forever. Use `runtime.NumGoroutine()` and `pprof` to detect leaks.
 4. **Prefer composition over orchestration.** Small functions that own one channel and one goroutine each are easier to reason about than one big function juggling 10 channels.
 5. **Document concurrency contracts.** “This method is safe to call from multiple goroutines” or “caller must hold the lock” goes in the godoc comment.
 6. **Test under load.** `go test -race -count=100` shakes out races that one run misses.
 7. **Avoid time.Sleep in tests.** It's flaky and slow. Use channels, `sync.WaitGroup`, or a fake clock.
 8. **Pin loop variables before Go 1.22.** `x := x` inside the loop body. After 1.22, the language does it for you.
-

Common gotchas

Gotcha	Symptom	Fix
Loop variable captured in goroutine (Go < 1.22)	All goroutines print the same value	Rebind <code>i := i</code> or pass as arg
Sending on closed channel	Panic	Only the sender closes; track close with a flag
Closing a nil channel	Panic	Check for nil before close
Forgotten <code>cancel()</code>	Goroutine and context leak	<code>defer cancel()</code> right after <code>WithContext</code>
<code>time.After</code> in hot loop	Excess allocations	Use <code>time.NewTimer</code> and <code>Reset</code>
<code>sync.WaitGroup.Add</code> inside goroutine	<code>Wait</code> returns early	Call <code>Add</code> before <code>go</code>
Map written without lock	Race, then runtime panic	Wrap in <code>RWMutex</code> or use <code>sync.Map</code>
Copying a <code>sync.Mutex</code>	Two locks instead of one	<code>go vet</code> catches it; always use pointer receivers
Nested locks in different orders	Deadlock	Define a global lock order
<code>select</code> with only <code>default</code>	CPU spin	Always have a blocking case
Unbuffered channel send with no receiver running yet	Hang	Start the receiver first, or buffer the channel
Returning a channel from a function that doesn't close it	Receiver hangs in <code>range</code>	Document who closes; usually the producer

Quick reference

Operation	Use
Hand-off one value, synchronize two goroutines	unbuffered channel
Decouple producer and consumer rates	buffered channel
Wait on multiple channel ops	<code>select</code>
Timeout a blocking op	<code>select + time.After</code> or <code>context.WithTimeout</code>
Cancel a tree of goroutines	<code>context.WithCancel</code>
Deadline across a request	<code>context.WithDeadline</code>
Protect shared state	<code>sync.Mutex</code>
Many readers, few writers	<code>sync.RWMutex</code>
Wait for N goroutines	<code>sync.WaitGroup</code>
Initialize once	<code>sync.Once</code>
Reusable temporary objects	<code>sync.Pool</code>
Concurrent counter	<code>atomic.Int64</code>
Fan-out with error propagation	<code>errgroup.Group</code>
Cap concurrency	buffered channel as semaphore
Drop on full instead of block	<code>select</code> with <code>default</code>
Disable a <code>select</code> branch	set its channel to nil
Detect races	<code>go test -race</code>

Go memory model: one-page summary

A read of a variable `r` is guaranteed to see a write `w` if either:

- `r` is sequenced after `w` in the same goroutine.
- `w` synchronizes-with some operation in `r`'s goroutine that is sequenced before `r`.

Synchronization operations:

- Channel send / receive on the same channel.
- Channel close / receive after close.
- `Mutex.Unlock` / next `Lock`.
- `Once.Do` returns.
- `WaitGroup.Done` / `Wait` returns.
-

`atomic` operations on the same variable. - `go` statement / start of new goroutine. -
`runtime.Goexit` / `defer` ordering within a goroutine.

Anything else: no guarantee. The compiler and CPU can reorder freely.